

CSE115L Project: Academic Transcript Generator in C

Objective

The goal of this project is to allow generation of customized and well-designed academic transcripts by performing necessary calculations and analysis using user-input data, and then optionally exporting the results into a printable and structured PDF file.

Tools Used

- **C Compiler:** GNU C Compiler (GCC) version 16.1.1
- **Build Tool:**
 - GNU Make on GNU/Linux and Unix-like systems
 - Manual on Windows
- **PDF Generation Library:** libharu [<https://github.com/libharu/libharu>]
- **Editor/IDE:**
 - Emacs on GNU/Linux
 - VSCode on Darwin (macOS)
 - Code::Blocks on Windows

Features

- **Support for multiple semesters:** Allows inclusion of multi-semester data without limits.¹
- **Robust data analysis:** On-the-fly TGPA and CGPA calculation.
- **Identity processing:** Allows the user to enter basic profile data such as name & ID.
- **ID Decoding (NSU-style):** Automatic department and degree inferring from ID.
- **PDF Generation:** Cross-platform PDF creation support, powered by libharu.
- **Command-line interface (CLI):** User-friendly interface and prompt.
- **C99 standard:** Codebase follows GNU C99 standard for maximum compatibility.

1. There is no practical limit in the implementation; however, this does not stop other limits from existing. Physical, hardware and operating system limits still apply.

Example Runs

The following example command was executed on a Windows 11 x86_64 machine, using the Code::Blocks IDE, which invokes gcc (the compiler) and the compiled program atsgen. Appropriate compiler and linker flags were added and libraries were copied correctly before running the compilation process. The instructions for adding the appropriate flags and libraries are given at the end of the output below.

```
Starting interactive transcript generation.
>>> Enter your name: Ar Rakin
>>> Enter your ID: 2611576642
>>> Enter your date of birth (format dd/MM/YYYY): 20/01/2008
>>> Enter semester number (e.g. 261; 0 to end): 253
Semester set to Fall 2025
Courses are listed below, please enter the appropriate course IDs indicated inside square brackets ('[]').
**Enter 0 to stop adding courses and finalize the semester record.**
```

```
[1]: CSE115 - Programming Language I
```

```

[2]: CSE115L - Programming Language I Lab
[3]: CSE173 - Discrete Mathematics
[4]: CSE215 - Programming Language II
[5]: CSE215L - Programming Language II Lab
[6]: CSE225 - Data Structures & Algorithms
[7]: CSE225L - Data Structures & Algorithms Lab
[8]: MAT116 - Pre-calculus
[9]: MAT120 - Calculus I
[10]: MAT125 - Linear Algebra & Analytical Geometry
[11]: ENG103 - Intermediate Composition
[12]: ENG105 - Advanced Composition
[13]: ENG111 - Public Speaking

>>> Enter course ID to add: 1
>>> Enter marks for [1] CSE115 - Programming Language I: 94
Course added: CSE115 - Programming Language I [Marks: 94/100]

>>> Enter course ID to add: 2
>>> Enter marks for [2] CSE115L - Programming Language I Lab: 100
Course added: CSE115L - Programming Language I Lab [Marks: 100/100]

>>> Enter course ID to add: 8
>>> Enter marks for [8] MAT116 - Pre-calculus: 97
Course added: MAT116 - Pre-calculus [Marks: 97/100]

>>> Enter course ID to add: 11
>>> Enter marks for [11] ENG103 - Intermediate Composition: 89
Course added: ENG103 - Intermediate Composition [Marks: 89/100]

>>> Enter course ID to add: 0

>>> Enter semester number (e.g. 261; 0 to end): 261
Semester set to Spring 2026
Courses are listed below, please enter the appropriate course IDs indicated inside square brackets ('[]').
**Enter 0 to stop adding courses and finalize the semester record.**

[1]: CSE115 - Programming Language I
[2]: CSE115L - Programming Language I Lab
[3]: CSE173 - Discrete Mathematics
[4]: CSE215 - Programming Language II
[5]: CSE215L - Programming Language II Lab
[6]: CSE225 - Data Structures & Algorithms
[7]: CSE225L - Data Structures & Algorithms Lab
[8]: MAT116 - Pre-calculus
[9]: MAT120 - Calculus I
[10]: MAT125 - Linear Algebra & Analytical Geometry
[11]: ENG103 - Intermediate Composition
[12]: ENG105 - Advanced Composition
[13]: ENG111 - Public Speaking

>>> Enter course ID to add: 3
>>> Enter marks for [3] CSE173 - Discrete Mathematics: 93
Course added: CSE173 - Discrete Mathematics [Marks: 93/100]

>>> Enter course ID to add: 9
>>> Enter marks for [9] MAT120 - Calculus I: 98
Course added: MAT120 - Calculus I [Marks: 98/100]

>>> Enter course ID to add: 10
>>> Enter marks for [10] MAT125 - Linear Algebra & Analytical Geometry: 99
Course added: MAT125 - Linear Algebra & Analytical Geometry [Marks: 99/100]

>>> Enter course ID to add: 13
>>> Enter marks for [13] ENG111 - Public Speaking: 90
Course added: ENG111 - Public Speaking [Marks: 90/100]

>>> Enter course ID to add: 0

>>> Enter semester number (e.g. 261; 0 to end): 0
Student Name: Ar Rakin
Student ID: 2611576642
Date of Birth: 20 Jan 2008
Degree Conferred: Bachelor of Science in Computer Science & Engineering
CGPA: 3.86 (A-)

Semester: Fall 2025
TGPA: 3.79 (A-)
CGPA: 3.79 (A-)

```

ID	Course	Credits	Marks	GP	Grade
[1]	CSE115 - Programming Language I	3	94	4.00	A
[2]	CSE115L - Programming Language I Lab	1	100	4.00	A
[8]	MAT116 - Pre-calculus	3	97	4.00	A
[11]	ENG103 - Intermediate Composition	3	89	3.30	B+

Semester: Spring 2026

TGPA: 3.93 (A-)

CGPA: 3.86 (A-)

ID	Course	Credits	Marks	GP	Grade
[3]	CSE173 - Discrete Mathematics	3	93	4.00	A
[9]	MAT120 - Calculus I	3	98	4.00	A
[10]	MAT125 - Linear Algebra & Analytical Geometry	3	99	4.00	A
[13]	ENG111 - Public Speaking	3	90	3.70	A-

>>> Save as PDF? [y/N, filename=transcript.pdf]: y

Saving this transcript to: /home/rakinar2/Projects/atngen/transcript.pdf

Process completed with code 0. Press any key to continue...

Platform-specific Build Instructions

Windows

For Windows systems, a few additional steps need to be completed before the compilation can succeed. First, open the `atngen.c` file in Code::Blocks, and then edit the compiler flags: Settings > Compiler, then edit any checked compiler option set, and add:

- ☐ Compiler flags: `-I<full_project_path>\src -I<full_project_path>\include`
- ☐ Linker flags: `-L<full_project_path> -L<full_project_path>\src -lhpdf`

While replacing `<full_project_path>` with the actual absolute path to the project.

Ensure the file `libhpdf.dll` is in the `src` directory. It should be there by default. If not, please copy it manually from the project root directory to `src`.

Finally, to run the code: Build > Build & Run. This should build and run the project successfully.

GNU/Linux & Unix-like systems

To run the code on a GNU/Linux or any Unix-like machine, the `make` utility can be used. For example, on typical GNU/Linux systems, the following command:

```
$ make run-linux
```

Will compile and execute the program.

Significant Code Snippets

The following functions (only signatures are given) are the backbone of the program:

❑ `static void export_to_pdf(struct semester_records *records);`

This function initiates the export process of the given semester records into a well-formatted PDF output file. Quite a few other helper functions are present to complete this process step-by-step, which are called in this function both directly and indirectly. These helper functions are prefixed with `pdf_`.

❑ `static bool pdf_draw_header_top_data(HPDF_Doc pdf, HPDF_Page page, struct semester_records *records);`

A helper function called by `export_to_pdf()` to draw the student profile information right underneath the top heading.

❑ `static void pdf_draw_header(HPDF_Doc pdf, HPDF_Page page);`

Draws the header, which includes the title (stylized university name), slogan, address and the source of the transcript.

❑ `static void add_courses_interactive(int *added_courses, int *added_course_marks, int *added_course_count);`

Interactively takes course information input from user, and stores them in memory.

Contribution

Contributors are listed in alphabetical order.

Contributor	Contribution
Akib Elahi	Developed string processing, semester decoding and date processing features – 30%
Ar Rakin	Developed the PDF generation feature and integrated libharu – 40%
Momsad Hossain	Developed CGPA and weighted grade calculation features – 30%

Conclusion

This project aims to successfully demonstrate the core knowledge and application of C programming concepts into a real-world usable tool. It has some areas for improvement, such as integrating a remote database or handling rare edge cases.